

PPLE Agent

AI O&M Reliability Command Center

Struktur & Arsitektur Project

1. RINGKASAN

PPLE Agent adalah platform terintegrasi Predictive Maintenance (PdM) / Condition-Based Maintenance (CBM) untuk pembangkit listrik PLTU Jeranjang (3 × 25 MW).

Platform ini menggabungkan 6 Specialist AI Agent, Reliability Fusion Engine, estimasi RUL & Failure Probability, Safety Guardrail, integrasi Work Order/EAM, knowledge base training, dan voice assistant - dalam dua antarmuka pengguna yang berbagi satu core Python yang sama.

Vibration Agent
MCSA Agent
DGA Agent
Partial Discharge Agent
Tribology Agent
Thermal Agent

Tech stack:

Dashboard klasik	Streamlit (app.py + src/pages/)
Frontend modern	React 19 + Vite + Tailwind CSS (frontend/)
Backend API	FastAPI (api_server.py, Swagger di /docs)
Data	CSV/Excel + Word (.docx) + JSON config
Knowledge base	JSON terstruktur (Materi/) + keyword search + RAG opsional
(LangChain/FAISS)	
AI opsional	Google Gemini (src/llm_assistant.py) - selalu fallback ke
rule-based	

Prinsip inti: logika rule-based (threshold MCSA/ESA, status Normal/Alarm/High, dsb.) adalah *source of truth*. LLM adalah lapisan asistensi opsional yang tidak pernah menjadi dependency wajib.

2. PETA DIREKTORI

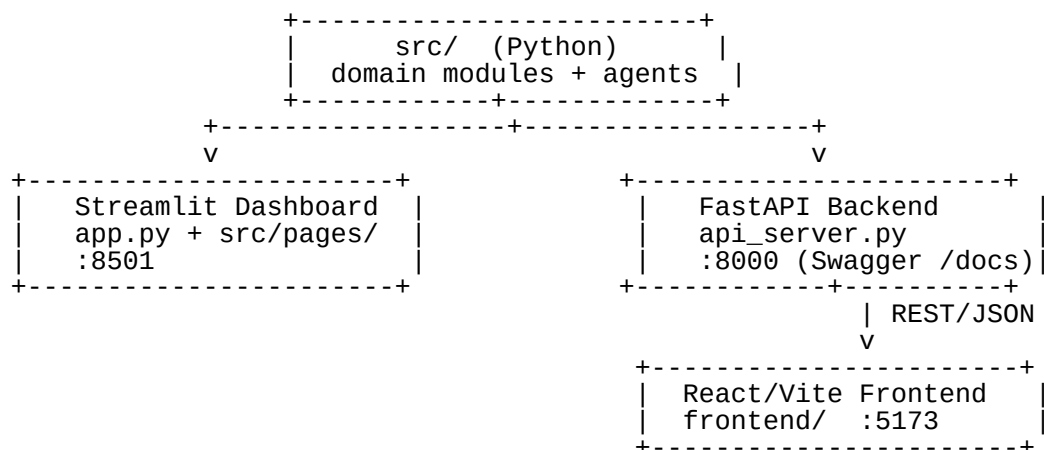
MCSA-main/	
+- app.py	# Entry point Streamlit dashboard
+- api_server.py	# Entry point FastAPI backend (42 endpoint REST)
+- run_server.py	# Helper menjalankan server
+- build_rag_index.py	# Build index semantik RAG (opsional, FAISS)
+- verify_app.py	# Verifikasi fungsional cepat (data, chatbot, PPT, Materi)
+- src/	# Core logic Python (dipakai Streamlit maupun FastAPI)
+- pages/	# Halaman UI Streamlit
+- components/	# Komponen UI Streamlit (sidebar, theme)
+- agents/	# 6 Specialist Agent + Fusion + Safety + Orkestrator
+- data_loader.py	# Resolusi path data, load & audit dataframe MCSA
+- standards.py	# Ambang batas & evaluasi standar MCSA/ESA
+- knowledge_processor.py	# Konversi upload (PDF/MD/DOCX/JSON/TXT) -> Materi v2 JSON
+- knowledge_retriever.py	# Keyword search Materi/ + config guidance
+- rag_engine.py	# Semantic search opsional (LangChain + FAISS)
+- llm_assistant.py	# Integrasi LLM opsional (Gemini), API-key
resolver	

```

|   +-- chatbot.py                # Logika chatbot rule-based
|   +-- docx_parser.py / docx_generator.py  # Parsing & generate laporan Word
|   +-- ppt_generator.py / ppt_theme.py / ppt_assets.py  # Generator laporan
PowerPoint
|   +-- vibration_data.py / tribology_data.py / thermal_data.py / dga_data.py  #
Domain data per modalitas
|   +-- rotorbar.py                # Kalkulasi rotor bar (sideband, health index)
|   +-- report_batches.py          # Manajemen batch upload laporan Word + audit
|   +-- time_filters.py            # Filter periode/tanggal
|   +-- analytics.py               # Analitik dashboard
|   +-- metadata.py                # Normalisasi unit/voltage/equipment
|   +-- agent_memory.py            # Riwayat percakapan (SQLite:
data/agent_memory.db)
|   +-- agent_cron.py              # Daemon background: siklus self-improvement
terjadwal
|   +-- equipment_mapping.json     # Mapping equipment
|   +-- utils.py
|
+-- frontend/                     # React/Vite SPA modern
|   +-- src/
|   |   +-- pages/                 # 11 halaman (routing di App.jsx)
|   |   +-- components/           # Komponen per domain (dga/, mcsa/, vibration/,
tribology/, thermal/, fusion/, health/, knowledge/, workorder/, common/)
|   |   +-- api.js                 # Helper pemanggilan FastAPI
|   |   (VITE_API_BASE_URL)
|   |   +-- App.jsx                # Router utama
|   +-- package.json              # Vite, React 19, Tailwind 4, oxlint (lint),
vite
|   +-- dist/                     # Hasil build produksi (gitignored)
|
+-- Materi/                       # Knowledge base - dokumen JSON v1/v2 (SOP,
manual, training)
|   +-- VIBRASI/ , TRIBOLOGY/ , Learning_agent/, ...
|
+-- data/                         # Data operasional (lihat detail §4)
|   +-- MCSA/                     # Sumber data utama MCSA/ESA
|   +-- DGA/ , vibrasi/           # Data modalitas lain
|   +-- learning/                 # State continuous-learning & env harness
|
+-- tests/                       # Unit test (unittest, bukan pytest)
+-- docs/                        # Dokumentasi tambahan (agents/,
plan_step_by_step.md)
+-- skills/                      # Skill packages agent (opsional)
|
+-- build.bat / run.bat / run_api.bat / run_frontend.bat / run_all.bat  # Skrip
Windows
+-- requirements.txt              # Dependency Python
+-- CLAUDE.md / AGENTS.md        # Instruksi kerja untuk AI coding agent
+-- README.md                    # Dokumentasi utama (Bahasa Indonesia)

```

3. ARSITEKTUR - DUA UI, SATU CORE



```
app.py      mengimpor SEMUA modul halaman di awal
           -> satu import rusak = seluruh app Streamlit down
           -> import halaman baru harus dijaga hati-hati

api_server.py  lapisan HTTP tipis di atas src/
           -> TIDAK boleh menduplikasi business logic
           -> semua kalkulasi/threshold tetap di src/

kedua UI    membaca sumber data yang SAMA
           (data/MCSA/mcsa_updated.csv)
           -> status equipment selalu konsisten di kedua antarmuka
```

```

USER (Voice/Chat/UI)
  |
  v
SubAgentCoordinator (subagent_coordinator.py)
  |
  +-----+-----+-----+-----+-----+
  |       |       |       |       |       |
  v       v       v       v       v       v
Vibration MCSA DGA PartialDischarge Tribology Thermal  <-
specialist_agents.py
  |       |       |       |       |
  +-----+-----+-----+-----+
  |
  v
Fusion Engine (fusion_engine.py)
Cross-domain evidence -> Health 0-100 -> RUL
  |
  v
Safety Guard (safety_guard.py)
Blokir Trip/Shutdown, wajib Human-in-the-Loop
  |
  v
Response tersintesis + Work Order

```

asset_graph.py	Knowledge graph hierarki plant: Plant -> Unit -> System -> Equipment -> Sub-component -> Sensor Stream
continuous_learning.py self_improvement.py	Siklus akurasi rekursif dengan guardrail never-regress
env_harness.py	Lingkungan simulasi untuk pembelajaran agent

agent_cron.py	Daemon yang menjalankan siklus self_improvement secara terjadwal. Setiap cycle error-safe - gagal tidak mematikan daemon.
agent_memory.py	Menyimpan riwayat percakapan (SQLite: data/agent_memory.db)

data/MCSA/Report MCSA.xls	Sumber Excel fallback
data/MCSA/mcsa_updated.csv	DATA AKTIF - hasil sync Word + edit manual + merge
data/MCSA/Laporan/UNIT {1,2,3,COMMON}/...	Struktur laporan Word per unit/voltage

```

data/MCSA/Laporan/uploads/YYYY/MM/DD/batch-HHMMSS-xxxxxx/
    Arsip tiap batch upload
    + manifest.json (checksum, status
    preview/karantina, audit_events)

data/MCSA/config/
    CONFIG ASLI:
    - equipment_master.json
    - esa_mcsa_guidance.json
    - thresholds_default.json
    - thresholds_motor_override.json
    - thresholds_site_override.json

data/MCSA/backup/
    Backup CSV otomatis (timestamped)
    sebelum overwrite atomic

data/DGA/ , data/vibrasi/
    Data modalitas DGA & Vibrasi
    (gambar, PDF, asset)

data/learning/
    State continuous-learning
    & env harness

Resolusi path data: get_data_path() di src/data_loader.py membaca MCSA_DATA_DIR (default
data/) dan otomatis fallback ke subfolder data/MCSA/ bila file dicari tidak ada di root.

Environment variables (.env.example):

MCSA_DATA_DIR      Direktori data persisten          default: ./data
MCSA_MAX_BACKUPS   Jumlah backup CSV dipertahankan      default: 5 (min 1)
WORK_ORDERS_FILE   File JSON status Work Order runtime default: data runtime
lokal
VITE_API_BASE_URL  Base URL FastAPI untuk frontend React default:
http://localhost:8000
GEMINI_API_KEY     Opsional, fitur AI                    default: kosong

.env tidak dimuat otomatis oleh aplikasi - kecuali src/llm_assistant.py, yang secara khusus
mem-parsing .env untuk API key AI.

Urutan resolusi API key:

argumen -> Streamlit session state -> st.secrets
        -> environment variable -> .env

```

5. KNOWLEDGE BASE (Materi/)

Dua skema JSON didukung:

```

// Skema v1 - list halaman
[{"page": 1, "content": "..."}]
// atau
{"pages": [{"page": 1, "content": "..."}]}

// Skema v2 - artikel + sections
{
  "id": "mcsa-bearing-fault",
  "title": "Analisis MCSA untuk Bearing Fault",
  "tags": ["MCSA", "Motor", "Bearing"],
  "level": "intermediate",
  "source": "Internal Training",
  "language": "id",
  "sections": [
    {"id": "intro", "heading": "Pendahuluan", "content": "..."}
  ]
}

```

```

src/knowledge_processor.py    Mengonversi upload (PDF/MD/DOCX/JSON/TXT)
                             -> v2 JSON

src/knowledge_retriever.py   Keyword search (selalu tersedia, tanpa
                             dependency tambahan). Juga meng-index
                             panduan/threshold dari data/MCSA/config/.

```

src/rag_engine.py

Semantic search OPSIONAL (LangChain + FAISS + sentence-transformers), khusus Materi/VIBRASI/ dan Materi/TRIBOLOGY/. Selalu fallback ke keyword search bila dependency opsional tidak terpasang.

Catatan penting:

Setelah menulis/menghapus file knowledge, WAJIB panggil `load_knowledge_base(force_reload=True)` - index adalah cache module-level global.

Untuk file Materi berukuran besar, pencarian dibatasi ke file terpilih agar tetap ringan; disarankan memecah per topik.

6. ALUR SYNC LAPORAN WORD

1. Aktifkan Mode Edit
 v
 2. Halaman "Sync Laporan Word" -> pilih .docx/.docm -> "Siapkan preview"
 v
 3. Sistem memeriksa equipment, tanggal, metadata, jumlah parameter, status tiap file
 v
 4. "Konfirmasi dan perbarui dashboard"
 v
 File valid -> masuk mcsa_updated.csv
 File bermasalah -> diarsipkan sebagai karantina
 v
 5. Tiap upload disimpan sebagai batch terpisah dengan manifest.json untuk audit
 v
 6. Equipment + tanggal yang sama -> batch terbaru jadi data aktif TANPA menghapus arsip sebelumnya
 v
 7. Dashboard otomatis diarahkan ke tanggal/equipment dari batch baru setelah commit
-

7. PARAMETER DOMAIN (Rotor Bar & Performance Summary)

Rotor Bar:

Upper/Lower Sideband
Rotorbar Health (RB Hlt Index)
Se Fund / Se Harm
Rotorbar Level %
Rotorbar Severity Level (1-4)
Status 3 warna: Normal / Alarm / High

Rekalkulasi saat load data HANYA untuk data terbaru per equipment (menjaga performa).

Performance Summary (field teks Bahasa Indonesia, prefix "Ringkasan Kinerja - ..."):

Kesimpulan
Faktor Daya
Arus
Tegangan
Beban
Koneksi Fasa
Rotor
Stator

Air-gap Rotor/Stator
Distorsi Harmonik
Misalignment/Unbalance
Bearing

Ambang batas evaluasi (standards.py & specialist agents):

Parameter	Normal	Alarm	High
Rotor Bar sideband (Upper/Lower)	< -54 dB	-54 s/d -45 dB	>= -45 dB
Unbalance Tegangan (dev. % fase)	<= 1%	> 1%	> 2%
Unbalance Arus (dev. % fase)	<= 5%	> 5%	> 10%
THD Tegangan (ref. IEEE 519)	<= 5%	> 5%	> 8%

8. FRONTEND REACT - PETA HALAMAN & API

Routing (frontend/src/App.jsx):

/	Dashboard	Ringkasan fleet
/fusion	ReliabilityCommandCenter	Health matrix, RUL, multi-agent collaboration hub
/twin	DigitalTwinWorkspace	Digital twin asset
/health	AssetHealth	Detail kesehatan aset
/mcsa	MCSAWorkspace	Data & kalkulator MCSA/rotor bar
/vibration	VibWorkspace	Data getaran, ISO 10816 evaluator
/tribology	TribologyWorkspace	Data oli, kalkulator kondisi oli
/thermal	ThermalWorkspace	Data thermal/IR
/dga	DGAWorkspace	Dissolved Gas Analysis, Duval Triangle
/knowledge	KnowledgeWorkspace	Pencarian & manajemen Materi
/workorders	WorkOrderCenter	Work Order & EAM

Backend api_server.py - 42 endpoint REST (Swagger /docs):

```
/api/health
/api/summary
/api/equipment
/api/equipment/{equipment_name}
/api/rotorbar/calculate

/api/agents/specialists
/api/agents/collaborate
/api/agent/chat chatbot + LLM opsional

/api/reliability/fleet
/api/reliability/fusion/{equipment_name}
/api/reliability/diagnose

/api/workorders
/api/workorders/approve

/api/vibration/*
/api/dga/*
/api/tribology/*
/api/thermal/*

/api/materi
/api/materi/search
/api/reports/ppt

/api/upload/dga
/api/upload/vibration

/api/agent/self-improve
/api/agent/improvement-status
/api/agent/predictions
/api/agent/learn-from-history
/api/learning/harness-status
/api/learning/rigger-cycle
```

/api/learning/evaluate-harness

/api/skills/learned-patterns

/api/skills/teach

/api/skills/benchmarks

Lint frontend: oxlint (bukan eslint)
npm --prefix frontend run lint

Build: npm --prefix frontend run build
-> bundle tunggal ~460 KB (~116 KB gzip)
-> belum menggunakan route-based code splitting

9. TESTING & VERIFIKASI

Framework: unittest (bukan pytest)
harus dijalankan dari root repo
(test meng-import "from src..." dan
"from api_server import app")

test_api_server.py memakai FastAPI TestClient
-> tidak butuh server live

knowledge_retriever tests memverifikasi terhadap isi NYATA
Materi/*.json - mengganti nama/
menghapus file bisa merusak test

verify_app.py verifikasi fungsional cepat (load data,
chatbot, generate PPT, schema Materi);
butuh Report MCSA.xls atau mcsa_updated.csv

Semua unit test
python -m unittest discover -s . -p "test_*.py"

Test tunggal
python -m unittest tests.test_chatbot
python -m unittest tests.test_chatbot.ChatbotTests.test_fuzzy_match_equipment

Verifikasi fungsional
python verify_app.py

Build penuh (venv -> pip install -> build frontend -> unit test ->
verify_app.py)
.\build.bat

10. MENJALANKAN APLIKASI

run.bat	Streamlit dashboard	http://localhost:8501
run_api.bat	FastAPI backend saja	http://localhost:8000 (/docs = Swagger)
run_frontend.bat	Vite dev server React	http://localhost:5173
run_all.bat	Backend + Frontend sekaligus	

Repo ini WINDOWS-FIRST:

Skip .bat membuat/menggunakan .venv dengan Python 3.11 (py -3.11).

Venv yang tercampur versi (mis. cp311 vs cp313) memicu layar perbaikan dependency di app.py - solusinya membuat ulang venv.

11. KEAMANAN & OPERASIONAL

Backup CSV otomatis bertimestamp di data/backup/
sebelum overwrite atomic; retensi via MCSA_MAX_BACKUPS.

Setiap batch upload laporan menyimpan manifest.json
(checksum, status, audit_events) untuk audit.

data/ TIDAK BOLEH disimpan di filesystem sementara saat
deployment - gunakan volume persisten dan arahkan MCSA_DATA_DIR.

API key tidak boleh di-hardcode; hanya lewat environment
variable atau .streamlit/secrets.toml (keduanya gitignored).

Safety Guardrail Agent memblokir perintah berisiko tinggi
(Trip/Shutdown) dan mewajibkan otorisasi manual engineer
(Human-in-the-Loop).

12. STATUS IMPLEMENTASI

Referensi: docs/plan_step_by_step.md

Arsitektur Multi-Agent & Specialist Sub-Agent - SELESAI

- [OK] SubAgentCoordinator (8 sub-agent)
- [OK] Endpoint kolaborasi backend
- [OK] Injeksi persona LLM
- [OK] UI kolaborasi frontend (badge sub-agent aktif,
accordion jejak kolaborasi)
- [OK] Suite test otomatis (test_subagent_coordinator.py)

*Dokumen ini dihasilkan dari eksplorasi langsung terhadap struktur kode per 2026-08-29. Untuk detail
command lengkap, lihat CLAUDE.md di root repo.*